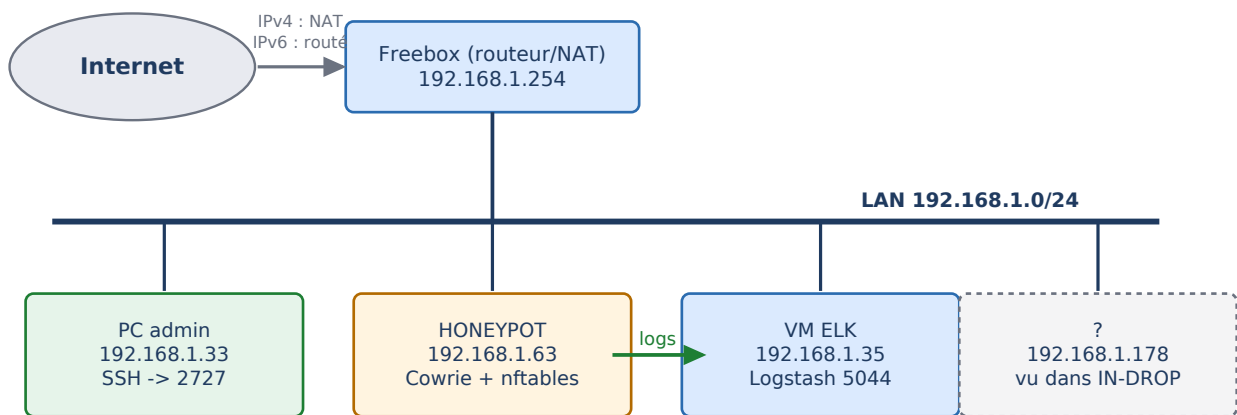


Correction du TP

Pare-feu d'un honeypot Cowrie — nftables

Corrigé établi à partir du fichier réellement présent sur la machine et des journaux du noyau, le 18 septembre 2026.



Seul flux sortant autorisé vers le LAN : 192.168.1.63 → 192.168.1.35:5044 (tcp)

Le laboratoire tel qu'il ressort de ton cahier des charges et des journaux de la machine.

Ce document est une correction, pas un jugement. Les erreurs commises ici — la *policy* par défaut, l'inversion source/destination, le DNS oublié — sont exactement celles que tout le monde fait en écrivant son premier pare-feu à état. Elles sont toutes instructives, et tes journaux en gardent la trace, ce qui permet de les corriger avec des preuves plutôt qu'avec des suppositions.

Synthèse

Voici ce qui est acquis et ce qui reste à travailler. Le détail de chaque point suit dans les sections suivantes.

Ce qui est juste

Point	Commentaire
Structure du fichier	Shebang, flush ruleset, define, table inet, trois chaînes : la charpente est correcte.
Usage des variables	\$ADMIN_IP, \$ELK_IP, \$SSH_PORT — bonne habitude, le fichier se relit et se modifie sans risque.
ct state invalid drop en tête	Placé au bon endroit, avant tout le reste.
Restriction du SSH par IP source	tcp dport \$SSH_PORT ip saddr \$ADMIN_IP accept : correct, et ça a fonctionné (voir la section « ce que tes journaux racontent »).
Règles de journalisation	log prefix ... limit rate dans les deux chaînes : c'est grâce à elles que ce corrigé a pu être écrit.
Réponse à l'exercice 2.1	Tu as trouvé le bug demandé : il manque ct state established, related accept dans la chaîne output. C'était bien la réponse.
Réponse à l'exercice 3.1	L'explication du filet de sécurité est juste et complète.

Ce qui est à revoir

Point	Gravité	Section
Aucune policy drop : le pare-feu n'interdit rien	Critique	§3 bug 1
Inversion saddr / daddr sur la règle ELK	Critique	§3 bug 2
Confusion sur le rôle de chaque chaîne (ports 80/443/53 en entrée)	Majeur	§3 bug 4
Chaîne forward vide et ouverte	Majeur	§3 bug 3
IPv6 : NDP bloqué, et adresse publique non prise en compte	Majeur	§3 bug 6
nftables.service désactivé : rien ne survit au redémarrage	Majeur	§3 bug 7
Signification de iif lo	À clarifier	§2, exo 2.1
Rôle de ct state established	À clarifier	§2, exo 2.1

En une phrase : le fichier est bien construit et bien commenté, mais dans son état actuel il **laisse tout passer**, parce qu'il manque la seule ligne qui transforme une liste d'autorisations en pare-feu : policy drop.

1. Correction des exercices

Tes réponses sont reprises en bleu, telles que tu les as écrites.

Exercice 0.1 — Un seul moteur ?

« J'ai le même retour pour « iptable » avec rien dedans, par contre j'imagine que on peut avoir des règles qu'on ne voit pas dans le legacy ou inversement mais les autres oui ? Je veux bien une explication. »

Ton intuition est juste. Voici le mécanisme exact.

Le noyau Linux contient **deux moteurs de filtrage distincts**, hérités de deux époques :

- x_tables — l'ancien, piloté par iptables-legacy ;
- nf_tables — le moderne, piloté par nft.

Sur Ubuntu 24.04, la commande iptables est en réalité un *traducteur* : elle accepte l'ancienne syntaxe mais écrit dans nf_tables. C'est ce que dit la mention (nf_tables) dans iptables -V. D'où le tableau :

Commande	Lit / écrit dans	Voit les règles de nft ?
nft list ruleset	nf_tables	oui
iptables -L	nf_tables (traduction)	oui , mais affichées en syntaxe iptables
iptables-legacy -L	x_tables	non — un univers séparé

Pourquoi c'est dangereux

Les deux moteurs s'accrochent aux **mêmes points du noyau** et sont consultés **tous les deux**. Un paquet doit donc être accepté par les deux pour passer. Conséquence : une règle DROP oubliée dans x_tables bloque le trafic, et tu peux relire ton nft list ruleset pendant des heures sans jamais la voir. C'est un des grands classiques du dépannage réseau sous Linux.

```
# le réflexe de diagnostic, à faire une fois :
sudo iptables-legacy -L -n -v          # doit être vide
sudo ip6tables-legacy -L -n -v         # idem, on oublie souvent l'IPv6
lsmod | grep -E 'ip_tables|nf_tables'  # quel moteur est réellement chargé
```

Exercice 1.1 (1) — L'architecture d'envoi des logs

« Je ne comprends pas. »

C'est la question la plus importante du TP, parce que la réponse détermine **dans quelle chaîne** tu dois écrire ta règle. Cowrie produit un journal au format JSON. Il existe deux façons de l'acheminer vers ELK :

	Chemin A — via Filebeat (ton choix)	Chemin B — envoi direct
Qui écrit	Cowrie écrit dans un fichier : log/cowrie.json	Cowrie utilise son greffon [output_elasticsearch]
Qui envoie	Filebeat , installé sur la machine hôte, lit le fichier	Cowrie lui-même , depuis l'intérieur du conteneur
Vers quoi	Logstash , port 5044 (protocole Beats)	Elasticsearch , port 9200 (HTTP)
Chaîne nftables concernée	output — c'est un processus de la machine qui ouvre la connexion	forward — c'est le conteneur qui ouvre la connexion, le paquet traverse la machine

Tu as renseigné ELK_PORT = 5044 : tu as donc choisi le **chemin A**. Ta règle a bien sa place dans la chaîne output. Retiens simplement que si un jour tu passes au chemin B, la règle devra migrer dans forward — sinon elle ne servira à rien.

À vérifier de ton côté : depuis la machine, 192.168.1.35 ne répond ni au ping ni sur le port 5044. Soit la VM ELK est éteinte, soit son adresse a changé, soit Logstash n'écoute pas encore. Tant que ce point n'est pas réglé, tu ne pourras pas valider la règle de sortie en conditions réelles.

Exercice 1.1 (2) — Le port d'exposition de Cowrie

« D'après le cahier des charges c'est le port 22 qui est utilisé par Cowrie. »

Correct sur le principe, avec une nuance de mise en œuvre qui compte. Cowrie ne doit **jamais tourner en root** : c'est un logiciel dont le métier est de se faire attaquer. Or les ports inférieurs à 1024 exigent des privilèges root. La pratique est donc :

- Cowrie écoute en interne sur un port haut, 2222, sans privilège ;
- on **redirige** 22 vers 2222, soit par Docker (-p 22:2222), soit par une règle de traduction d'adresse en nftables.

Le port 22 est donc le port *vu de l'extérieur*, pas celui sur lequel le programme écoute. Et comme tu as déplacé ton vrai SSH sur 2727, le 22 est effectivement libre : c'est un bon choix, et c'est précisément l'intérêt d'avoir déménagé le SSH d'administration.

Exercice 1.1 (3) — La sortie Internet

« Quand je le décide. »

C'est la bonne décision pour un honeypot, et elle est cohérente avec ta réponse à l'exercice 1.2. Elle a deux conséquences concrètes que tu dois assumer :

1. Pas de règle tcp dport {80, 443} dans le fichier permanent. Tu la charges ponctuellement, par un fichier séparé (fourni en §5).
2. unattended-upgrades ne pourra plus faire son travail. Soit tu l'acceptes et tu mets à jour à la main pendant tes fenêtres de maintenance, soit tu le désactives pour qu'il cesse d'échouer en silence tous les jours.

Exercice 1.2 — La question piège

« Les fichiers sont enregistrés dans un répertoire spécifique -> download_path = dl »

Exact, mais c'est la moitié de la réponse — et la moitié manquante est justement le piège.

Pour enregistrer le fichier dans dl/, Cowrie doit d'abord **le télécharger pour de vrai**. Le wget de l'attaquant est émulé côté shell, mais la requête HTTP sort réellement de ta machine, avec ton adresse IP publique.

Les conséquences :

- **Ton adresse IP apparaît** dans les journaux du serveur qui héberge le logiciel malveillant. Tu te signales à l'attaquant et à son infrastructure.
- Tu **télécharges volontairement du code malveillant** sur une machine de ton réseau domestique. C'est le but recherché quand on collecte des échantillons — c'est un risque inutile quand on apprend.
- Selon ton hébergeur ou ton opérateur, du trafic sortant vers des infrastructures malveillantes peut déclencher un signalement.

La bonne réponse en deux phrases : non, un honeypot d'apprentissage ne doit pas pouvoir sortir sur Internet ; Cowrie enregistrera alors l'URL tentée dans son journal — ce qui est l'information intéressante — sans jamais récupérer le fichier. On n'ouvre la sortie que si l'on fait de la collecte d'échantillons de façon délibérée, et de préférence depuis une machine isolée et jetable.

Exercice 2.1 — Les annotations ligne par ligne

Tu as annoté chaque ligne. Je reprends chacune de tes annotations.

« table inet filter » — table sur l'IPv4 et v6, je sais pas à quoi ça correspond

inet est une **famille d'adresses**. Elle indique quel type de trafic la table voit :

Famille	Voit	Quand l'utiliser
ip	IPv4 seulement	Anciennes configurations
ip6	IPv6 seulement	Anciennes configurations
inet	IPv4 et IPv6	Le choix par défaut aujourd'hui — une seule règle pour les deux
bridge / netdev	Trafic ponté / au ras de la carte réseau	Cas avancés

Le piège de inet, et tu es tombé dedans : la table voit les deux protocoles, mais une règle qui commence par `ip saddr` ne concerne que l'IPv4, et `ip6 saddr` que l'IPv6. En revanche `tcp dport 5044`, qui ne mentionne aucune version, s'applique **aux deux**. Écrire une table `inet` ne suffit donc pas à couvrir l'IPv6 : il faut y penser règle par règle.

« iif lo accept » — fais repartir la règle depuis le début, je ne comprends pas trop

C'est la seule erreur d'interprétation vraiment gênante de ta copie, parce qu'elle porte sur une notation qui revient partout. Il n'y a ici aucune boucle et aucun retour en arrière.

Notation	Se lit	Signifie
iif	<i>input interface</i>	l'interface par laquelle le paquet est entré
oif	<i>output interface</i>	l'interface par laquelle le paquet va sortir
lo	<i>loopback</i>	l'interface de bouclage : 127.0.0.1 et ::1, la machine qui se parle à elle-même

`iif lo accept` se lit donc : « **si ce paquet est arrivé par l'interface de bouclage, accepte-le** ». Et c'est vital : sur ta machine, `systemd-resolved` écoute sur 127.0.0.53. Chaque résolution de nom passe par le bouclage. Sans cette règle, avec une policy `drop`, tu perds la résolution DNS de toute la machine.

« ct state established,related accept » — les réponses se font automatiquement, sans passer par le pare-feu ?

Non, et la nuance est importante : **chaque paquet**, sans exception, traverse le pare-feu et est évalué règle par règle. Ce que fait cette règle, c'est **reconnaître d'un coup** tous les paquets qui appartiennent à une conversation déjà autorisée, au lieu de t'obliger à écrire une règle inverse pour chacune.

Sans elle, pour autoriser une simple requête web, il faudrait écrire : « laisse sortir le paquet vers le port 443 », puis « laisse entrer les paquets qui viennent du port 443 de n'importe quelle adresse » — ce qui revient à ouvrir grand la porte d'entrée. Le suivi de connexion supprime ce dilemme : c'est ce qui distingue un pare-feu **à état** d'un simple filtre.

« Il manque : ct state established,related accept » dans la chaîne output

Bonne réponse. C'était exactement le bug recherché.

Le scénario concret, si l'on applique ce ruleset pendant une session SSH :

1. Ton PC envoie un paquet SYN vers 192.168.1.63:2727. La chaîne input l'accepte : la règle `tcp dport 2727 ip saddr 192.168.1.33` matche.
2. `sshd` répond par un SYN-ACK. Ce paquet est **émis** par la machine : il traverse la chaîne output.
3. Dans output, aucune règle ne le matche : ce n'est ni du bouclage, ni du trafic vers l'ELK. Il tombe sur `policy drop`.
4. La réponse ne part jamais. **Ta session gèle et tu perds la machine** — d'où l'exercice 3.1.

Exercice 3.1 — Le filet de sécurité

« *Commande qui attend 2 minutes et supprime toutes les règles si on ne la kill pas ; ça nous permet d'avoir une sécu si on s'est bloqué. Pourquoi utiliser `bash -c` ?* »

L'explication du principe est juste et complète. Reste ta question sur `bash -c`, qui est une vraie subtilité du shell :

```
sudo sleep 120 && nft flush ruleset          # PIÈGE
# \_____/ \_____/
# exécuté en root exécuté par TON shell, sans sudo -> "Permission denied"

sudo bash -c 'sleep 120 && nft flush ruleset' # CORRECT
# \_____/
# un shell root exécute toute la chaîne, sudo compris
```

Le `&&` est interprété par le shell *appelant*. `sudo` ne s'applique qu'à la première commande. En passant toute la ligne à un shell lancé en root, l'ensemble hérite des privilèges. Les **apostrophes simples** sont indispensables : elles empêchent ton propre shell d'interpréter la chaîne avant de la transmettre.

L'alternative plus propre

```
sudo systemd-run --on-active=120 --unit=fw-rollback nft flush ruleset
# ... tester ...
sudo systemctl stop fw-rollback.timer      # annuler le filet une fois rassuré
```

Avantage décisif : la tâche est confiée à `systemd`, donc elle **survit à la fermeture du terminal**. Avec `&`, si ta session SSH tombe — ce qui est précisément le scénario contre lequel tu te protèges — le processus peut être tué avec elle, et le filet disparaît au pire moment.

2. Ce que tes journaux racontent

Tes règles log ont fonctionné : le noyau a conservé 777 lignes. Elles racontent précisément ce qui s'est passé pendant le TP. C'est la partie la plus utile de ce corrigé, parce qu'elle remplace les hypothèses par des faits.

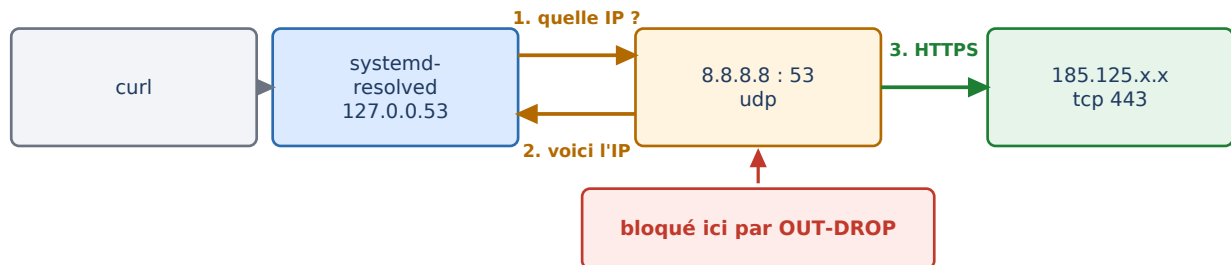
```
sudo dmesg | grep -E 'IN-DROP|OUT-DROP'
```

Nombre	Sens	Protocole	Ce que ça signifie
588	sortant	ICMPv6 types 135/136	Découverte de voisins IPv6 bloquée — IPv6 cassé (bug 6)
111	sortant	UDP port 53	Résolution DNS bloquée — la vraie cause de ton « le web ne marche pas »
54	sortant	TCP port 53	Idem, requêtes DNS en TCP vers fd0f:ee:b0::1
16	entrant	TCP port 2727	Tentatives SSH depuis 192.168.1.178 — ta règle a fait son travail
8	entrant	ICMPv6	Voisinage IPv6, côté entrant

Le diagnostic du « les requêtes web ne fonctionnent pas »

Tu as écrit dans ta copie : « les requêtes web ne fonctionnent pas, j'ai fait n'importe quoi à la fin, j'ai dupliqué les accept 443 et dns ». Les journaux montrent que le problème n'était **pas le web**, et donc que la correction ne pouvait pas être dans les règles 443.

Ce qui se passe réellement derrière un simple `curl https://archive.ubuntu.com`



Symptôme observé : « le web ne marche pas » — en réalité l'étape 3 n'est jamais atteinte.

Le paquet bloqué n'est pas du 443 : c'est de l'UDP/53.

Preuve dans tes logs : `OUT-DROP ... PROTO=UDP ... DPT=53 (111 paquets)`

Une requête web commence toujours par une requête DNS. Si le DNS est bloqué, le navigateur ou curl échoue avant même d'avoir tenté d'ouvrir le port 443.

Les 111 paquets UDP/53 refusés en sortie sont datés entre 9440 et 9516 secondes d'activité de la machine, soit une fenêtre de 76 secondes : exactement le moment où tu testais avec une `policy drop` active et sans règle DNS. Ce que ton système essayait d'atteindre :

Destination	Protocole	Paquets refusés
fd0f:ee:b0::1 (DNS de la Freebox, en IPv6)	TCP/53	54
fd0f:ee:b0::1	UDP/53	39
8.8.8.8	UDP/53	36
1.1.1.1	UDP/53	36

La leçon, et c'est la plus rentable du TP : le symptôme visible (« le web est cassé ») désigne rarement la règle fautive. Le journal, lui, donne le protocole et le port exacts du paquet refusé. **Lis le journal avant de modifier une règle** — sinon on ajoute des autorisations au hasard, ce qui affaiblit le pare-feu sans résoudre le problème.

Les tentatives SSH depuis 192.168.1.178

Seize paquets à destination du port 2727 ont été refusés, tous venant de 192.168.1.178. C'est une bonne et une moins bonne nouvelle :

- **Bonne** : ta règle `ip saddr $ADMIN_IP` fonctionne exactement comme prévu. Une machine du réseau qui n'est pas ton poste d'administration ne peut pas atteindre le SSH.
- **À éclaircir** : sais-tu ce qu'est 192.168.1.178 ? Si c'est un autre de tes appareils, tout va bien. Sinon, une machine de ton réseau cherche le SSH — ce qui mérite un coup d'œil.

```
ip neigh | grep 192.168.1.178      # adresse MAC -> constructeur de l'appareil
sudo nmap -sn 192.168.1.0/24      # inventaire du réseau local
```


3. Analyse de ton ruleset

Chaque bug est présenté ainsi : ce que tu as écrit, ce que fait réellement le noyau, et la correction.

Bug 1 — Aucune policy : le pare-feu n'interdit rien

Gravité : critique. C'est la clé de voûte manquante.

```
# ce que tu as écrit
type filter hook input priority 0;

# ce que le noyau a retenu (sortie de: sudo nft list ruleset)
type filter hook input priority filter; policy accept;
                                ^^^^^^^^^^^^^^^
```

Quand la policy n'est pas précisée, nftables retient accept. Ta chaîne est donc une liste d'autorisations... suivie d'une autorisation générale. Tout ce que tu n'as pas prévu passe quand même.



le log s'affiche... et le paquet passe quand même



le log dit ce que tu as oublié d'autoriser

À gauche ce que fait ton fichier aujourd'hui, à droite ce qu'il devrait faire. Les règles sont identiques : seule la dernière ligne change tout.

Effet pervers : tes règles log continuent d'écrire « IN-DROP » et « OUT-DROP » dans le journal. **Le préfixe ment** : log journalise mais ne rend aucun verdict, et le paquet poursuit son chemin jusqu'à la *policy*, qui l'accepte. C'est pour cela que tout « fonctionne » sur ta machine en ce moment.

Correction :

```
type filter hook input priority filter; policy drop;
```

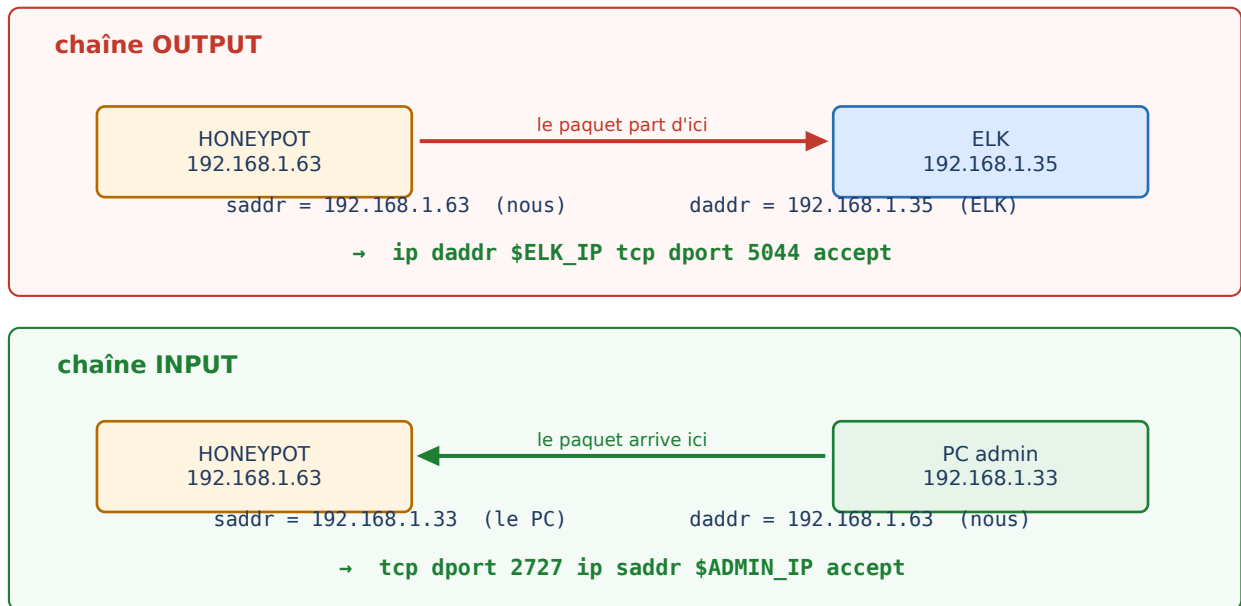
Au passage, `priority filter` est préférable à `priority 0` : c'est le même nombre, mais le mot-clé reste juste si les conventions changent, et il se relit mieux.

Bug 2 — Inversion de la source et de la destination

Gravité : critique. La règle la plus importante de ton pare-feu — celle qui justifie tout le TP — ne peut jamais matcher.

```
# ton fichier, chaîne output :
tcp dport $ELK_PORT ip saddr $ELK_IP accept
                                ^^^^^^^^^^^^^^^
                                « le paquet VIENT de 192.168.1.35 »
```

Dans la chaîne output, le paquet est **émis par ta machine** : sa source est toujours 192.168.1.63. La condition `ip saddr 192.168.1.35` est donc impossible à satisfaire. Avec une `policy drop`, tes logs ne seraient jamais partis vers ELK.



Le même couple d'adresses s'inverse selon la chaîne. C'est le point de vue de la machine qui compte, jamais celui du service distant.

Le réflexe à acquérir : avant d'écrire une règle, demande-toi « *qui ouvre la connexion ?* ». Si c'est ma machine → chaîne output, le correspondant est en `daddr`. Si c'est le correspondant → chaîne input, il est en `saddr`.

Correction :

```
ip daddr $ELK_IP tcp dport $ELK_PORT ct state new accept
```

Le `ct state new` ajouté n'est pas obligatoire, mais il est plus précis : il dit « autorise l'ouverture de cette connexion ». La suite du dialogue est déjà couverte par la règle `established, related` placée plus haut.

Bug 3 — La chaîne forward, vide et grande ouverte

```
chain forward {  
    type filter hook forward priority filter;    # -> policy accept  
}
```

Cette chaîne concerne tout le trafic qui **traverse** la machine sans lui être destiné : c'est-à-dire, à partir du moment où tu installeras Docker, **l'intégralité du trafic de Cowrie** — celui des attaquants vers le conteneur, et celui du conteneur vers ton réseau. La laisser en `accept` revient à annuler l'objectif du TP : empêcher un honeypot compromis d'aller voir tes autres machines.

Correction : `policy drop`, plus le trio habituel (`invalid`, `established`, `journalisation`). Le détail Docker est traité en §6.

Bug 4 — Des ports ouverts en entrée par confusion de sens

Gravité : majeure — et c'est la conséquence directe du diagnostic raté du §2.

```
# ton fichier, chaîne input :
tcp dport 53 accept
udp dport 53 accept
tcp dport {443, 80} accept
```

En chaîne input, dport désigne un port **de ta machine**. Ces trois lignes se lisent donc :

- « n'importe qui sur Internet peut interroger un serveur DNS hébergé sur cette machine » ;
- « n'importe qui sur Internet peut atteindre un serveur web hébergé sur cette machine ».

Ce n'est évidemment pas ce que tu voulais : tu cherchais à réparer tes requêtes *sortantes*. Les réponses à tes propres requêtes n'ont besoin d'aucune de ces règles — elles sont déjà couvertes par `ct state established,related accept`.

Aujourd'hui ces règles n'ouvrent rien de concret, parce qu'aucun service n'écoute sur ces ports. Mais elles constituent une **bombe à retardement** : le jour où tu lances un conteneur avec un serveur web, il devient joignable depuis Internet sans que tu aies rien décidé. Un pare-feu doit énoncer une intention, pas des vestiges de dépannage.

Correction : supprimer ces trois lignes de la chaîne input, et corriger la sortie dans output en nommant les serveurs DNS :

```
# dans output – on autorise, mais seulement vers les résolveurs déclarés
ip daddr { 8.8.8.8, 1.1.1.1 } udp dport 53 accept
ip daddr { 8.8.8.8, 1.1.1.1 } tcp dport 53 accept
```

Bug 5 — Les ports de Cowrie dans la mauvaise chaîne

```
tcp dport {22, 23} accept          # dans input
```

L'intention est juste, l'emplacement non. Une fois Cowrie lancé dans Docker, les paquets des attaquants sont redirigés vers l'adresse interne du conteneur : ils **traversent** la machine et passent par forward, jamais par input. Cette règle ne servira donc à rien — et, plus déroutant, Cowrie fonctionnera quand même, parce que Docker installe ses propres règles. Voir §6, c'est le piège principal du sujet.

Bug 6 — IPv6 : le point le plus important pour un honeypot

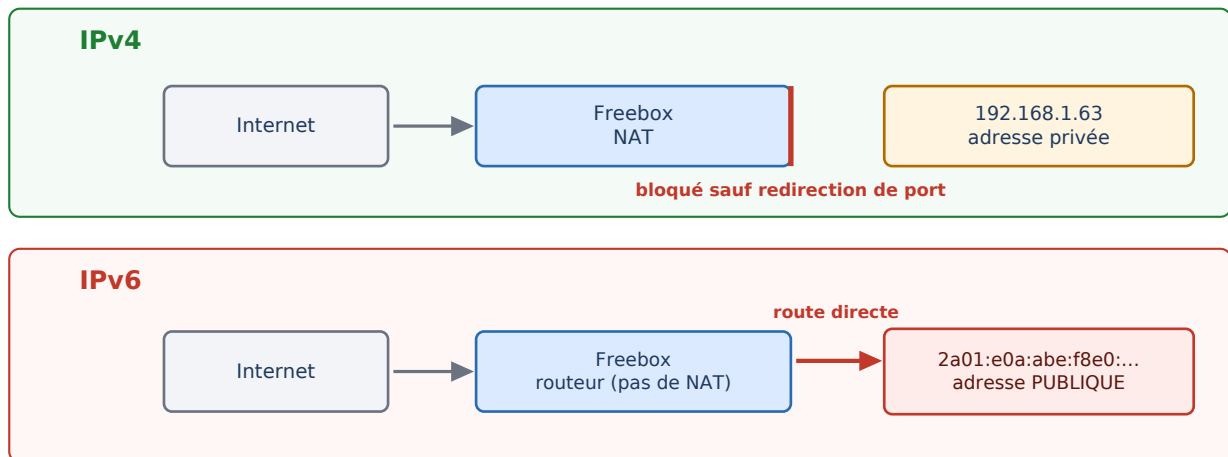
Gravité : majeure. Deux constats indépendants.

a. La découverte de voisins est bloquée

588 paquets ICMPv6 de types 135 et 136 refusés en sortie. Ce sont les messages *neighbor solicitation* et *neighbor advertisement* : l'équivalent IPv6 d'ARP. Sans eux, la machine ne peut plus associer une adresse IPv6 à une adresse matérielle, et l'IPv6 cesse de fonctionner. Ta chaîne input les autorisait ; ta chaîne output les avait oubliés.

b. Ta machine possède une adresse IPv6 publique

```
$ ip -6 -br addr
wlp3s0    UP    2a01:e0a:abe:f8e0:a2d3:7aff:feb5:58ab/64    fe80::a2d3:7aff:feb5:58ab/64
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  adresse publique, routable
```



En IPv4 la box te protège par effet de bord, grâce au NAT. En IPv6 il n'y a pas de NAT : seul le pare-feu décide.

À retenir absolument. En IPv4 tu es derrière le NAT de la Freebox : rien n'entre sans redirection de port explicite. En IPv6, ta machine est **directement routable depuis Internet**. Le seul rempart est ton pare-feu, et il est aujourd'hui en policy accept. Pour un honeypot — une machine dont le métier est d'être attaquée — c'est le point de vigilance numéro un.

Correction : une policy drop qui s'applique bien aux deux familles (c'est le cas avec inet), et un jeu de règles ICMPv6 complet **dans les deux chaînes**. Contrairement à l'IPv4, on ne peut pas filtrer l'ICMPv6 à la serpe : il porte des fonctions vitales (voisinage, découverte de routeur, taille maximale de paquet).

Bug 7 — Rien ne survit au redémarrage

```
$ systemctl is-enabled nftables
disabled
$ systemctl is-active nftables
inactive
```

Ton ruleset a été chargé à la main par nft -f. Il vit uniquement en mémoire. Au prochain redémarrage, la machine repart **sans aucune règle**. C'est d'autant plus trompeur que /etc/nftables.conf existe et paraît en place.

Correction : une fois le fichier validé et testé,

```
sudo systemctl enable --now nftables
```

À faire **en dernier**, et jamais avant d'avoir vérifié que tu conserves l'accès SSH — sinon un redémarrage suffit à te verrouiller dehors définitivement.

Points mineurs

Ce que tu as écrit	Remarque
ip saddr 192.168.1.0/24 icmp type { ... destination-unreachable, time-exceeded }	Ces deux messages d'erreur viennent surtout d' Internet , pas du réseau local : ils signalent une route cassée ou un paquet trop grand. Les restreindre au LAN peut provoquer des blocages difficiles à diagnostiquer. Ils sont de toute façon classés related par le suivi de connexion.
log prefix "IN-DROP " sans level	Le niveau par défaut est warn, ce qui remonte dans la console. level info est plus discret et suffit.
Ordre des règles	ct state established,related accept gagne à être la première règle après invalid : c'est par elle que passe l'immense majorité des paquets, et chaque règle placée avant est évaluée inutilement pour chacun d'eux.
udp dport 123 accept sans destination	Fonctionne, mais autorise le NTP vers n'importe quelle adresse. On peut le restreindre aux serveurs effectivement utilisés (systemctl status systemd-timesyncd les affiche).

4. Le ruleset corrigé

Voici le fichier complet. Il a été **vérifié syntaxiquement** sur ta machine avec `nft -c -f`, sans être appliqué. Les choix suivent tes réponses : sortie Internet fermée, logs vers 192.168.1.35:5044, administration depuis 192.168.1.33 seulement.

```
#!/usr/sbin/nft -f
flush ruleset

define ADMIN_IP      = 192.168.1.33
define SSH_PORT      = 2727
define ELK_IP        = 192.168.1.35
define ELK_PORT      = 5044
define LAN           = 192.168.1.0/24
define DNS_V4        = { 8.8.8.8, 1.1.1.1 }
define COWRIE_PORTS = { 22, 23 }

table inet filter {
    chain input {
        type filter hook input priority filter; policy drop;

        ct state invalid drop
        ct state established,related accept
        iif lo accept

        ip saddr $LAN icmp type { echo-request, echo-reply } accept
        icmpv6 type { echo-request, echo-reply, nd-neighbor-solicit,
                    nd-neighbor-advert, nd-router-solicit, nd-router-advert,
                    packet-too-big, time-exceeded, parameter-problem,
                    destination-unreachable } accept

        tcp dport $SSH_PORT ip saddr $ADMIN_IP ct state new accept
        tcp dport $COWRIE_PORTS ct state new accept

        log prefix "IN-DROP " level info limit rate 5/minute
    }

    chain forward {
        type filter hook forward priority filter; policy drop;
        ct state invalid drop
        ct state established,related accept
        log prefix "FWD-DROP " level info limit rate 5/minute
    }

    chain output {
        type filter hook output priority filter; policy drop;

        ct state invalid drop
        ct state established,related accept
        oif lo accept

        icmpv6 type { nd-neighbor-solicit, nd-neighbor-advert,
                    nd-router-solicit, destination-unreachable,
                    packet-too-big, time-exceeded, parameter-problem } accept
        ip saddr $LAN icmp type echo-request accept

        ip daddr $ELK_IP tcp dport $ELK_PORT ct state new accept

        ip daddr $DNS_V4 udp dport 53 accept
        ip daddr $DNS_V4 tcp dport 53 accept
        udp dport 123 accept

        log prefix "OUT-DROP " level info limit rate 5/minute
    }
}
```

Ce qui a changé, et pourquoi

Modification	Raison
<code>policy drop</code> sur les trois chaînes	Bug 1 — sans elle, rien n'est filtré.
<code>ip daddr \$ELK_IP</code> au lieu de <code>saddr</code>	Bug 2 — la règle ne pouvait jamais matcher.
Chaîne <code>forward</code> fermée et journalisée	Bug 3 — c'est par là que passera tout Cowrie.
Suppression des <code>dport 80/443/53</code> en entrée	Bug 4 — ils ouvraient des services inexistants.
DNS en sortie restreint aux résolveurs déclarés	Bug 4 — on autorise un flux nommé, pas « le port 53 ».
ICMPv6 complet dans output aussi	Bug 6 — sans NDP sortant, l'IPv6 ne fonctionne plus.
ICMP entrant limité à <code>echo-request/reply</code>	Les messages d'erreur sont déjà couverts par <code>related</code> .
<code>ct state new</code> sur les ouvertures	Précision : on autorise l'ouverture, pas un paquet isolé.
Aucune règle 80/443 en sortie	Ta réponse à l'exercice 1.1 (3) : sortie Internet à la demande.
<code>level info</code> sur les logs	Évite de polluer la console.

Note sur les ports de Cowrie. La règle `tcp dport $COWRIE_PORTS` est conservée en `input` pour le cas où tu lancerais Cowrie *hors* Docker. Dès que tu passes par un conteneur, c'est la chaîne `forward` et la chaîne `DOCKER-USER` qui prennent le relais (§6).

5. Mise en place, sans perdre la machine

L'ordre des opérations est la partie qui compte. Chaque étape est réversible tant que la précédente a été respectée.

Étape 1 — Deux terminaux et un filet

terminal A : on laisse une session SSH ouverte et on n'y touche plus

```
# terminal B :  
sudo systemd-run --on-active=180 --unit=fw-rollback nft flush ruleset
```

Étape 2 — Vérifier la syntaxe sans appliquer

```
sudo nft -c -f /etc/nftables.conf      # -c : contrôle seul, rien n'est chargé
```

Étape 3 — Appliquer et tester immédiatement

```
sudo nft -f /etc/nftables.conf  
sudo nft list ruleset | head -20      # vérifier que "policy drop" apparaît bien
```

Puis, **dans le terminal A** — celui qui était déjà ouvert — tape simplement une commande. Si elle répond, ta session a survécu. Ensuite seulement, ouvre une **troisième** connexion SSH depuis ton PC : c'est elle qui prouve qu'une *nouvelle* connexion est encore possible.

Une session SSH déjà ouverte survit grâce à `ct state established`, même si la règle d'ouverture est fautive. **Seule une nouvelle connexion prouve que le pare-feu est correct.** C'est le test que l'on oublie, et c'est celui qui coûte cher.

Étape 4 — Batterie de tests

Commande	Depuis	Attendu
<code>ssh -p 2727 ubuserv@192.168.1.63</code>	PC admin (.33)	passé
<code>ssh -p 2727 ...</code>	une autre machine du LAN	timeout
<code>getent hosts archive.ubuntu.com</code>	la machine	résout (DNS autorisé)
<code>curl -sI https://archive.ubuntu.com</code>	la machine	échoue — voulu : sortie fermée
<code>nc -zv 192.168.1.35 5044</code>	la machine	passé (si ELK est allumé)
<code>nc -zv 192.168.1.50 22</code>	la machine	échoue + ligne OUT-DROP
<code>ping6 -c2 2001:4860:4860::8888</code>	la machine	voir NDP : ne doit plus saturer le journal

Étape 5 — Annuler le filet, puis rendre permanent

```
sudo systemctl stop fw-rollback.timer  
sudo systemctl enable --now nftables  
sudo reboot      # la seule preuve qui vaille
```

Le fichier de maintenance (sortie Internet à la demande)

Conformément à ta réponse à l'exercice 1.1 (3), la sortie web n'est pas dans le fichier permanent. Crée `/etc/nftables.d/maintenance.nft` :

```
#!/usr/sbin/nft -f  
# Ouvre la sortie web. À charger le temps d'une mise à jour, puis recharger le fichier normal.  
table inet filter {  
    chain output {  
        ip daddr != 192.168.1.0/24 tcp dport { 80, 443 } ct state new accept  
    }  
}
```

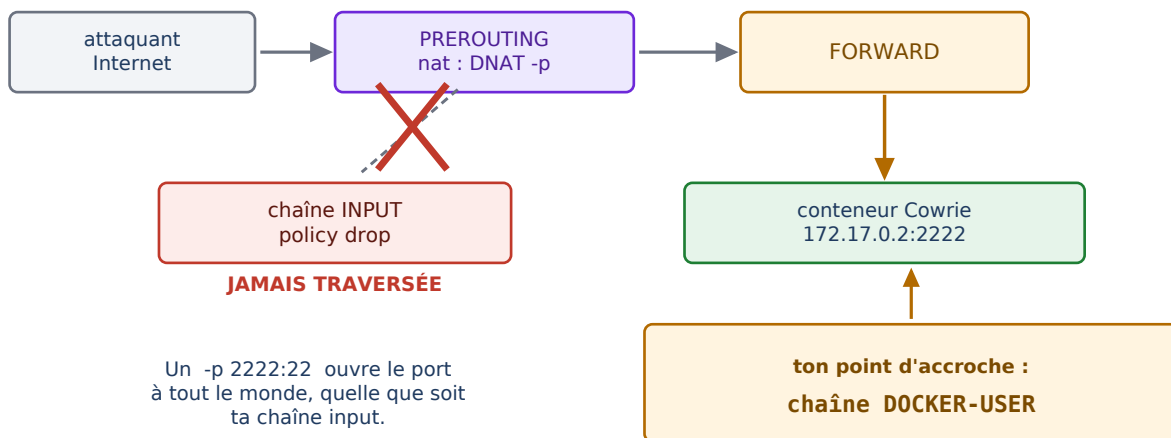


```
sudo nft -f /etc/nftables.d/maintenance.nft # ouvrir
sudo apt update && sudo apt upgrade
sudo nft -f /etc/nftables.conf # refermer (le flush ruleset remet tout à zéro)
```

Le ip daddr != 192.168.1.0/24 est important : il ouvre le web vers l'extérieur sans rouvrir de chemin vers tes autres machines du réseau local.

6. Ce qui t'attend : Docker

Tu n'as pas encore installé Docker — docker est absent de la machine. C'est une bonne chose : tu abordes l'étape suivante avec un pare-feu propre. Voici ce qui va se passer.



Règle à retenir : ce qui entre dans un conteneur passe par FORWARD, pas par INPUT.

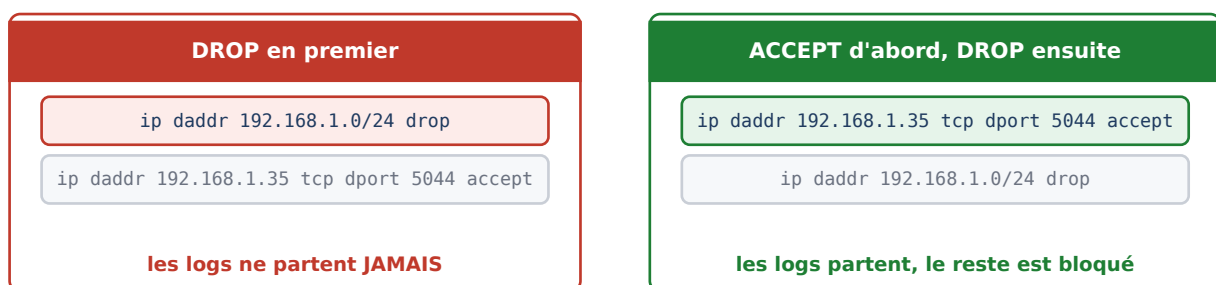
Un port publié par Docker ne traverse jamais la chaîne input : il est redirigé très tôt, puis routé vers le conteneur en passant par forward.

Au démarrage, Docker installe ses propres règles et active le routage (`net.ipv4.ip_forward=1`). Un `-p 22:2222` crée une redirection d'adresse qui s'applique **avant** ta chaîne input : le port devient joignable quelle que soit ta politique d'entrée.

Docker laisse toutefois un point d'accroche explicite : la chaîne `DOCKER-USER`, évaluée avant ses propres règles. C'est là, et seulement là, que tes décisions sur le trafic des conteneurs doivent s'écrire.

```
# autoriser Cowrie à parler à ELK, lui interdire le reste du réseau local
sudo iptables -I DOCKER-USER 1 -d 192.168.1.35 -p tcp --dport 5044 -j ACCEPT
sudo iptables -I DOCKER-USER 2 -d 192.168.1.0/24 -j DROP
```

L'ordre compte : la première règle qui matche gagne



L'ordre est vital : une règle DROP large placée avant l'autorisation ELK bloquerait aussi les logs.

Le piège de la persistance : Docker recrée ses chaînes à chaque démarrage du service et tes ajouts disparaissent. Trois approches, à comparer dans la suite du TP : le paquet `iptables-persistent` ; un `ExecStartPost` ajouté à `docker.service` ; ou une chaîne `nftables` branchée sur `forward` avec une priorité inférieure à celle de Docker, ce qui la fait passer avant.

7. Ce qu'il reste à faire

#	Action	Pourquoi
1	Corriger <code>/etc/nftables.conf</code> avec le fichier du §4, en suivant la procédure du §5	C'est le cœur du TP
2	Vérifier l'état de la VM ELK (192.168.1.35 ne répond pas)	Sans elle, la règle de sortie ne peut pas être validée
3	Identifier 192.168.1.178	Une machine du réseau a cherché ton SSH
4	<code>systemctl enable nftables</code> après validation	Sinon tout disparaît au redémarrage
5	Corriger <code>gateway4: 10.0.0.1</code> dans <code>netplan</code>	Adresse hors de ton sous-réseau
6	Protéger <code>50-cloud-init.yaml</code> (<code>chmod 600</code>)	Il contient la clé Wi-Fi en clair et il est lisible par tous
7	Installer Docker puis écrire les règles <code>DOCKER-USER</code>	§6
8	Décider du sort d' <code>unattended-upgrades</code>	Il échouera en silence avec la sortie fermée

Le mot de la fin. Ce TP t'a fait rencontrer, en une séance, les quatre erreurs qui font perdre une machine à distance : oublier la *policy*, inverser source et destination, se tromper de chaîne, et oublier que l'IPv6 existe. Tu les as désormais rencontrées **avec les journaux pour les prouver**, ce qui vaut beaucoup mieux que de les avoir lues quelque part. La compétence à retenir n'est pas la syntaxe de `nftables` : c'est le réflexe de lire le journal avant de toucher à une règle.